

GLM

Bhavani Chalasani - 2022101014

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.2.0
## v forcats    1.0.1      v stringr   1.6.0
## v ggplot2    4.0.1      v tibble    3.3.1
## v lubridate  1.9.5      v tidyr     1.3.2
## v purrr      1.2.1
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(stringr)
library(lubridate)
library(car)
```

```
## Loading required package: carData
##
## Attaching package: 'car'
##
## The following object is masked from 'package:dplyr':
##
##     recode
##
## The following object is masked from 'package:purrr':
##
##     some
```

```
library(nortest)
library(effects)
```

```
## Registered S3 method overwritten by 'lme4':
##   method      from
##   na.action.merMod car
## lattice theme set by effectsTheme()
## See ?effectsTheme for details.
```

```
library(e1071)
```

```
##
```

```

## Attaching package: 'e1071'
##
## The following object is masked from 'package:ggplot2':
##
##     element

library(marginaleffects)
library(betareg)

get_time_of_day <- function(ts) {
  time_ist <- as_datetime(ts / 1000, tz = "Asia/Kolkata")
  hour <- hour(time_ist)
  case_when(
    hour >= 6 & hour < 16 ~ "Day time",
    # hour >= 12 & hour < 17 ~ "Afternoon",
    hour >= 16 & hour < 20 ~ "Evening",
    TRUE ~ "Night"
  )
}

raw_dataset <- read_csv(
  "sentence_memorability_data.csv",
  na = c("", "NA", "N/A"),
  show_col_types = FALSE
)

data <- raw_dataset %>%
  arrange(participant_ID, Timestamp) %>%
  mutate(
    Accuracy_IR = suppressWarnings(as.numeric(Accuracy_IR)),
    Accuracy_WR = suppressWarnings(as.numeric(Accuracy_WR)),
    prev_IR = lag(Accuracy_IR)
  ) %>%
  group_by(participant_ID) %>%
  summarise(
    Total_Repeats = sum(isRepeat == TRUE & Event == "Sentence shown", na.rm = TRUE),
    Total_Denom = sum(is.na(isRepeat) & Event == "Sentence shown", na.rm = TRUE),
    Hits = sum(Event == "IR pressed" & isRepeat == TRUE & Accuracy_IR == 1, na.rm = TRUE),
    False_Alarms = sum(Event == "IR pressed" & is.na(isRepeat), na.rm = TRUE),

    Hit_Rate = if_else(Total_Repeats > 0, Hits / Total_Repeats, 0),
    False_Alarm_Rate = if_else(Total_Denom > 0, False_Alarms / Total_Denom, 0),
    IR_Memorability = Hit_Rate - False_Alarm_Rate,

    WR_correct = sum(Event == "WR pressed" & Accuracy_WR == 1 & prev_IR == 1, na.rm = TRUE),
    WR_total = sum(Event == "WR pressed" & prev_IR == 1, na.rm = TRUE),
    WR_Accuracy = if_else(WR_total > 0, WR_correct / WR_total, 0),

    IR_Presses = sum(Event == "IR pressed", na.rm = TRUE),
    WR_Presses = sum(Event == "WR pressed", na.rm = TRUE),
    avg_RT_IR = mean(Reaction_time_IR, na.rm = TRUE),
    avg_RT_WR = mean(Reaction_time_WR, na.rm = TRUE),
    avg_timestamp = mean(Timestamp, na.rm = TRUE),

```

```

    .groups = "drop"
  ) %>%
  mutate(
    Time_of_Day = get_time_of_day(avg_timestamp)
  )

data <- data %>%
  mutate(
    Time_of_Day = factor(Time_of_Day,
                        levels = c("Day time", "Evening", "Night"))
  )

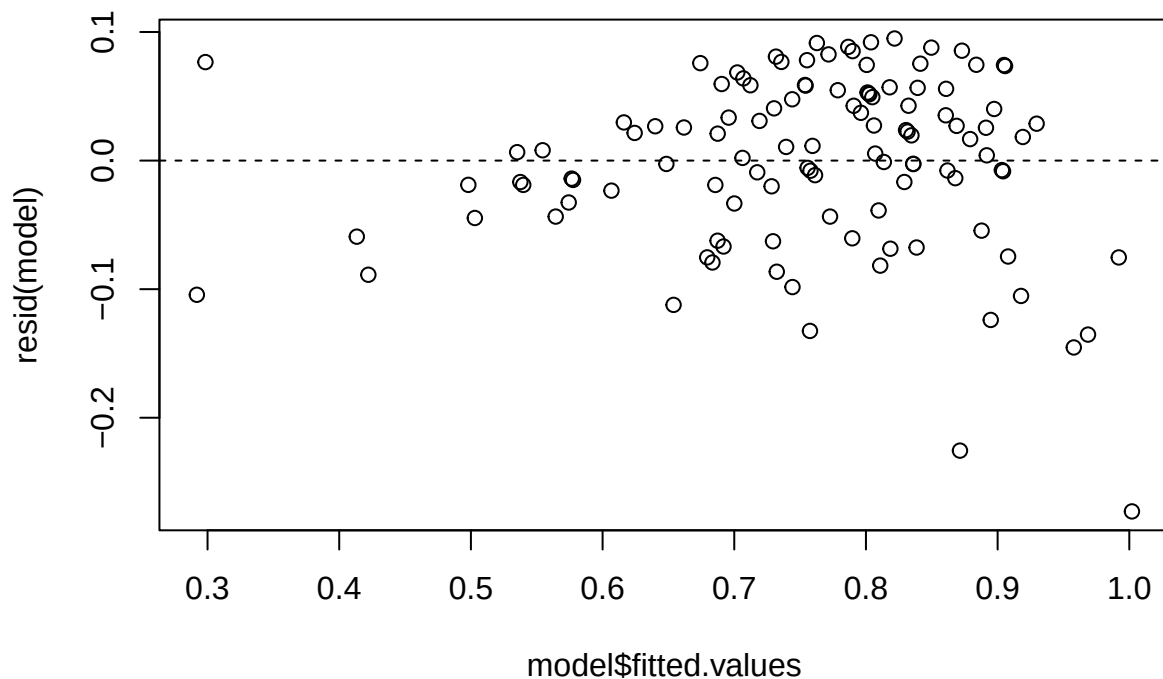
# Testing assumptions - IR memorability

model <- lm(IR_Memorability ~ avg_RT_IR + IR_Presses + Time_of_Day, data = data)

# Linearity and homoskedasticity

plot(model$fitted.values, resid(model))
abline(h = 0, lty = 2)

```



```

ncvTest(model)

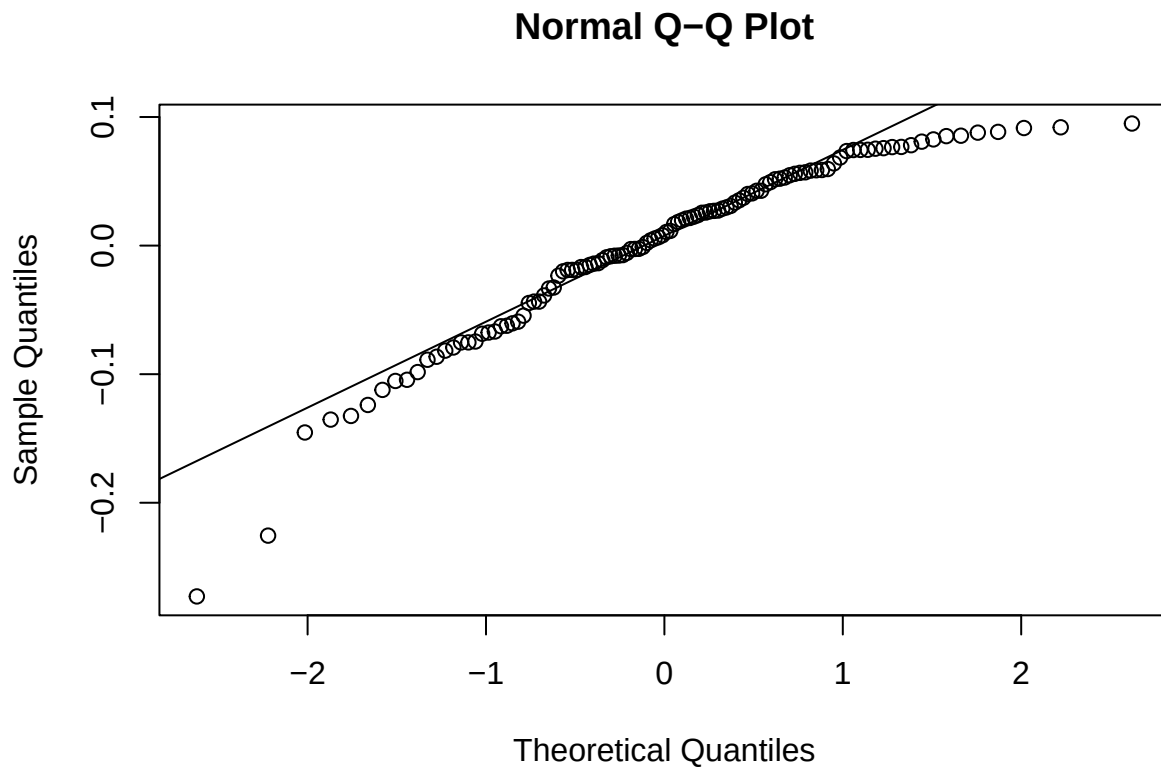
## Non-constant Variance Score Test
## Variance formula: ~ fitted.values

```

```
## Chisquare = 11.18524, Df = 1, p = 0.0008245
```

```
# Normality
```

```
qqnorm(resid(model))  
qqline(resid(model))
```

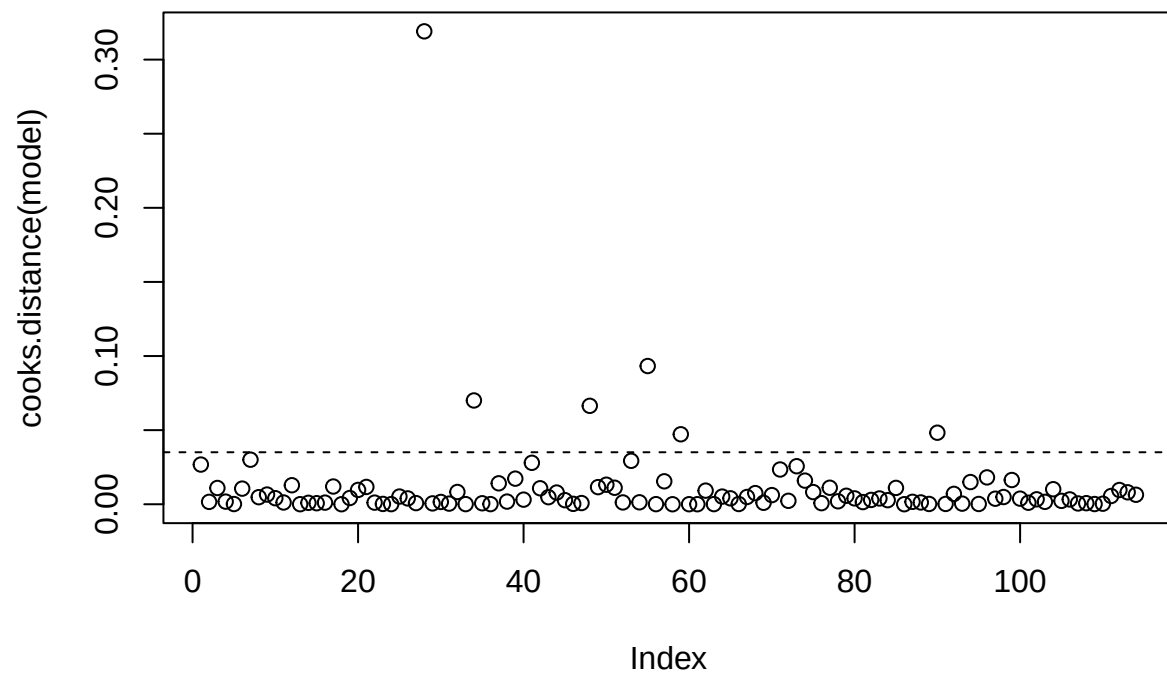


```
residuals_model <- residuals(model)  
ad.test(residuals_model)
```

```
##  
## Anderson-Darling normality test  
##  
## data: residuals_model  
## A = 1.6174, p-value = 0.0003517
```

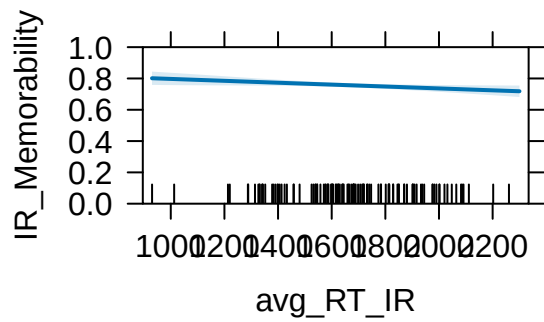
```
# Outliers
```

```
plot(cooks.distance(model))  
abline(h = 4/nrow(data), lty = 2)
```

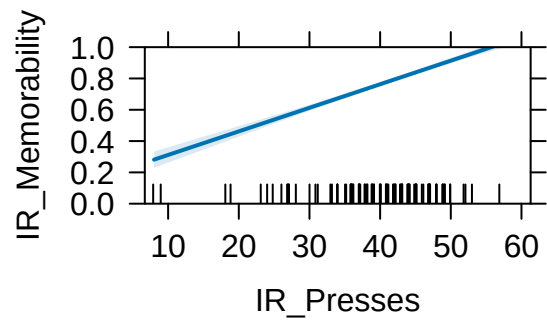


```
# Transforming the data because the GLM assumptions were violated  
plot(allEffects(model), ylim = c(0, 1))
```

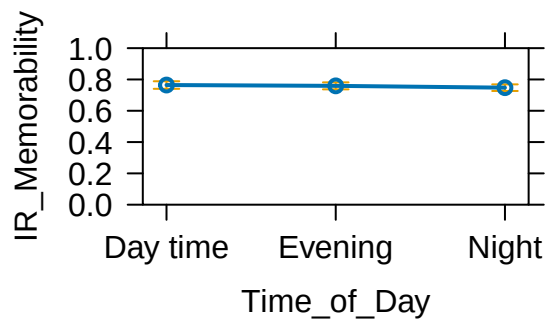
avg_RT_IR effect plot



IR_Presses effect plot

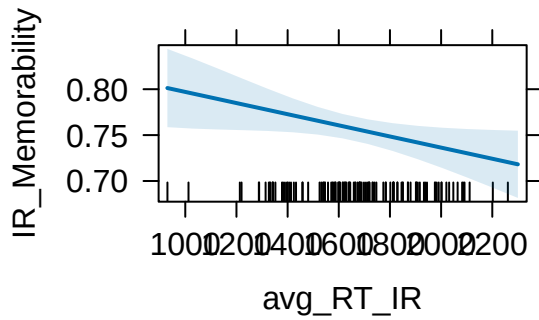


Time_of_Day effect plot

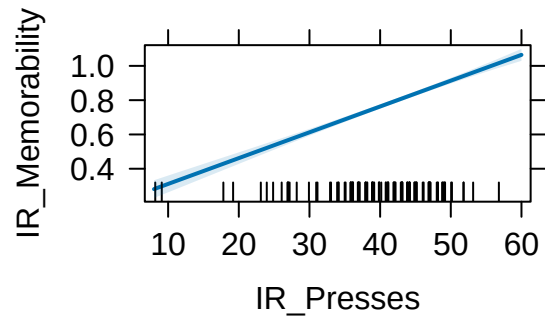


```
plot(allEffects(model))
```

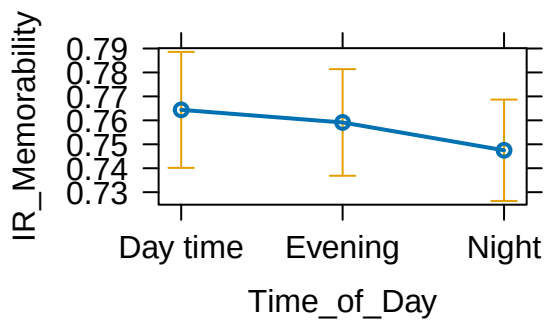
avg_RT_IR effect plot



IR_Presses effect plot

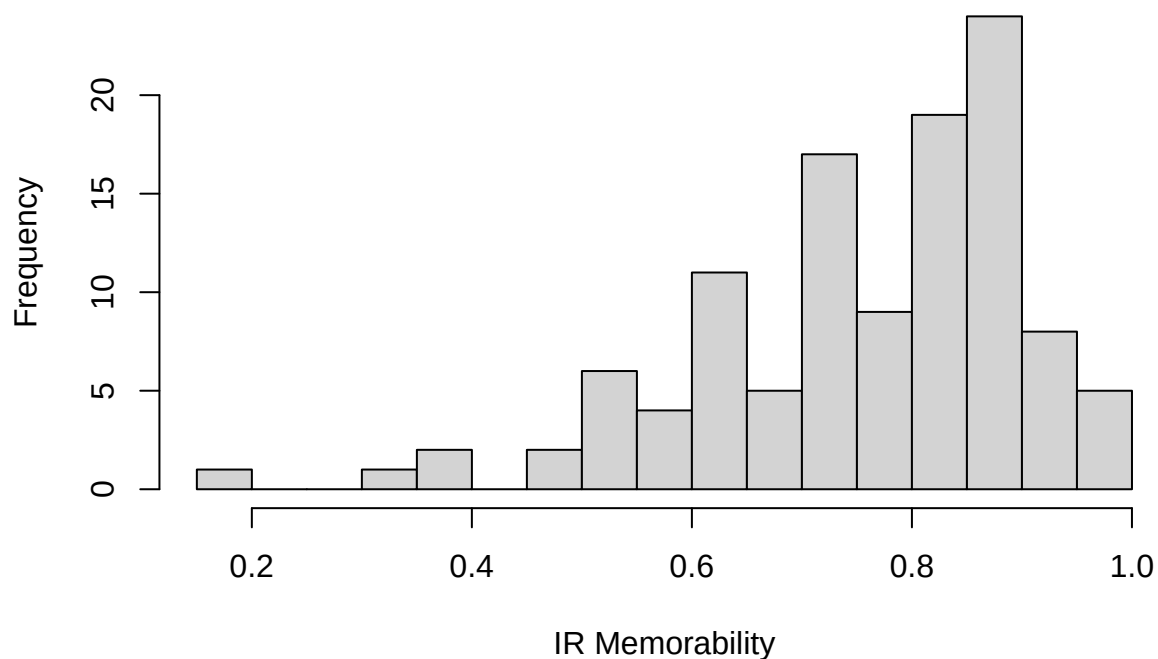


Time_of_Day effect plot



```
hist(data$IR_Memorability,  
      breaks = 20,  
      main = "Distribution of IR Memorability",  
      xlab = "IR Memorability")
```

Distribution of IR Memorability



```
# Fixing linearity
data <- data %>%
  mutate(
    log_RT_IR = log(avg_RT_IR)
  )

# Fixing homoskedasticity and normality - checking skewness to see which transform to apply
cat("Checking skew:\n")
```

```
## Checking skew:
```

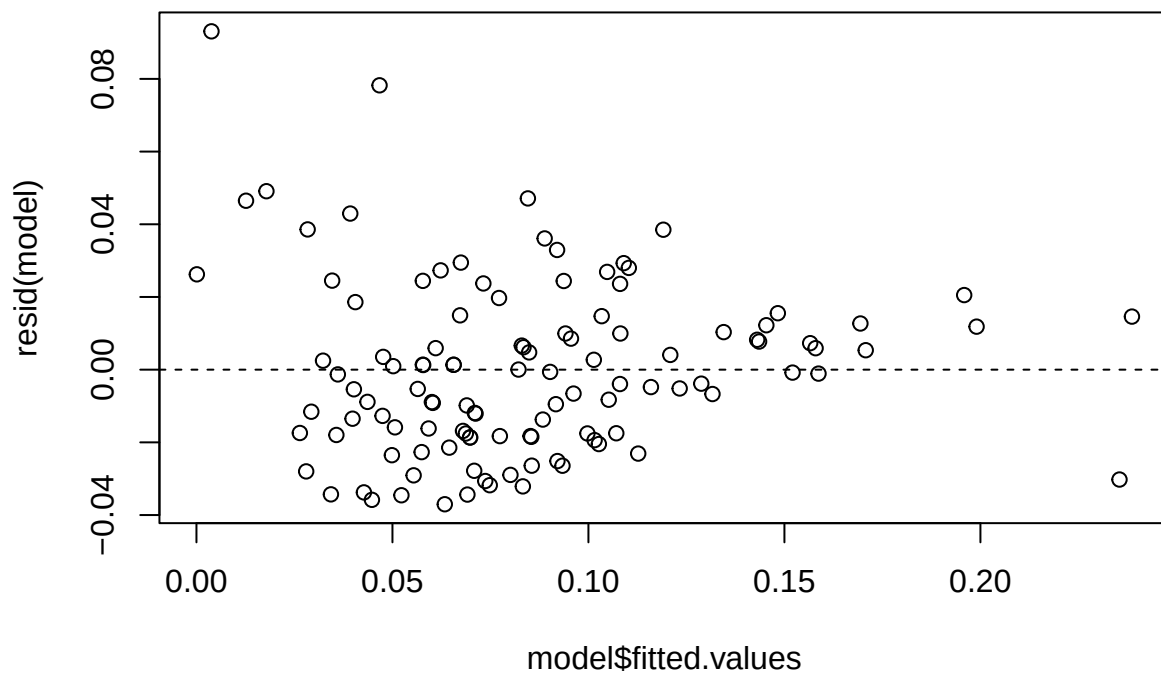
```
skewness(residuals_model)
```

```
## [1] -1.092266
```

```
K <- max(data$IR_Memorability, na.rm = TRUE) + 1
data <- data %>%
  mutate(
    IR_transformed = log10(K - IR_Memorability)
  )

model <- lm(IR_transformed ~ log_RT_IR + IR_Presses + Time_of_Day, data = data)

plot(model$fitted.values, resid(model))
abline(h = 0, lty = 2)
```

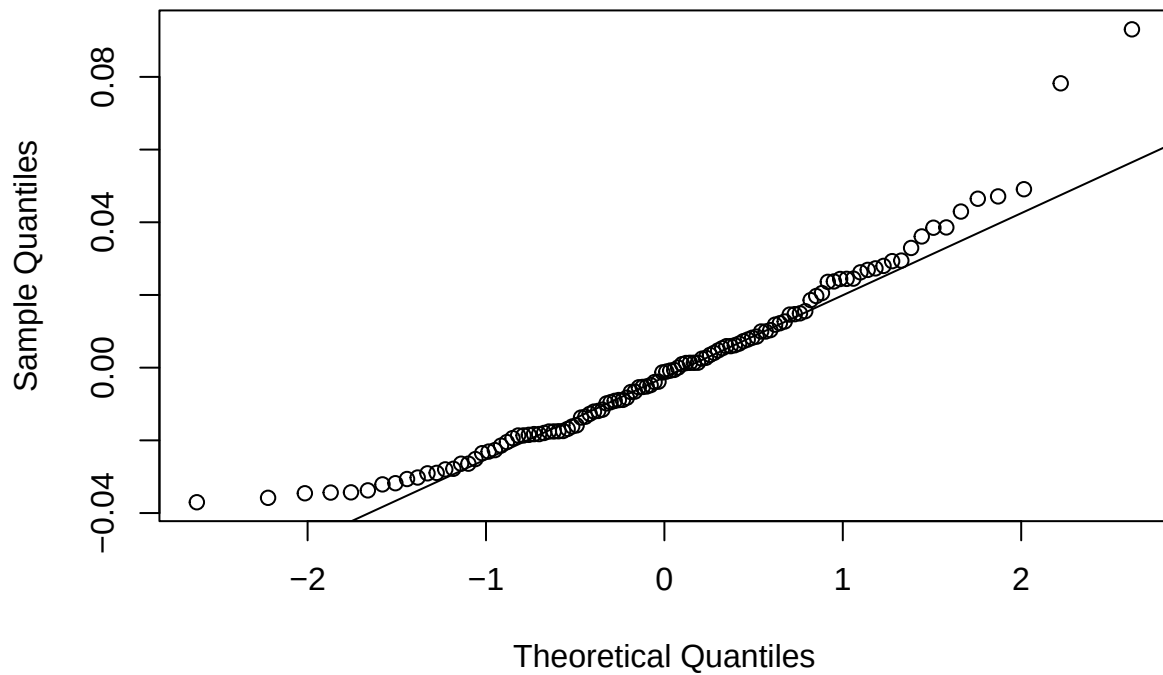
```
ncvTest(model)
```

```
## Non-constant Variance Score Test  
## Variance formula: ~ fitted.values  
## Chisquare = 17.02081, Df = 1, p = 3.6972e-05
```

```
# Normality
```

```
qqnorm(resid(model))  
qqline(resid(model))
```

Normal Q-Q Plot

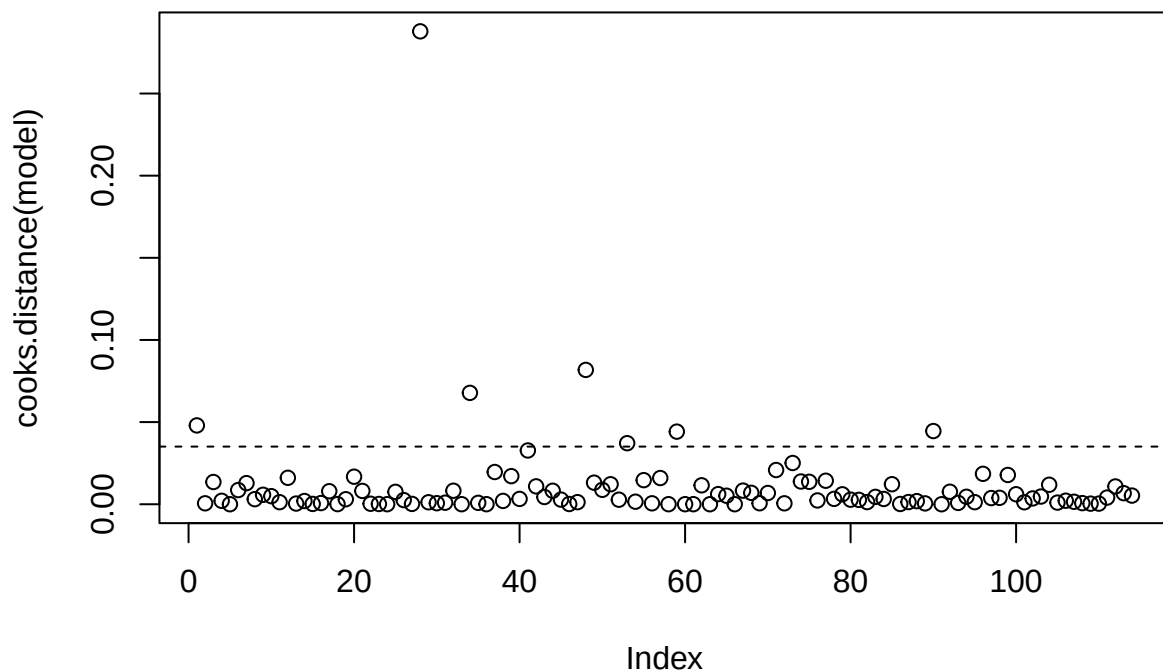


```
residuals_model <- residuals(model)
ad.test(residuals_model)
```

```
##
## Anderson-Darling normality test
##
## data: residuals_model
## A = 0.98837, p-value = 0.01267
```

```
# Outliers
```

```
plot(cooks.distance(model))
abline(h = 4/nrow(data), lty = 2)
```



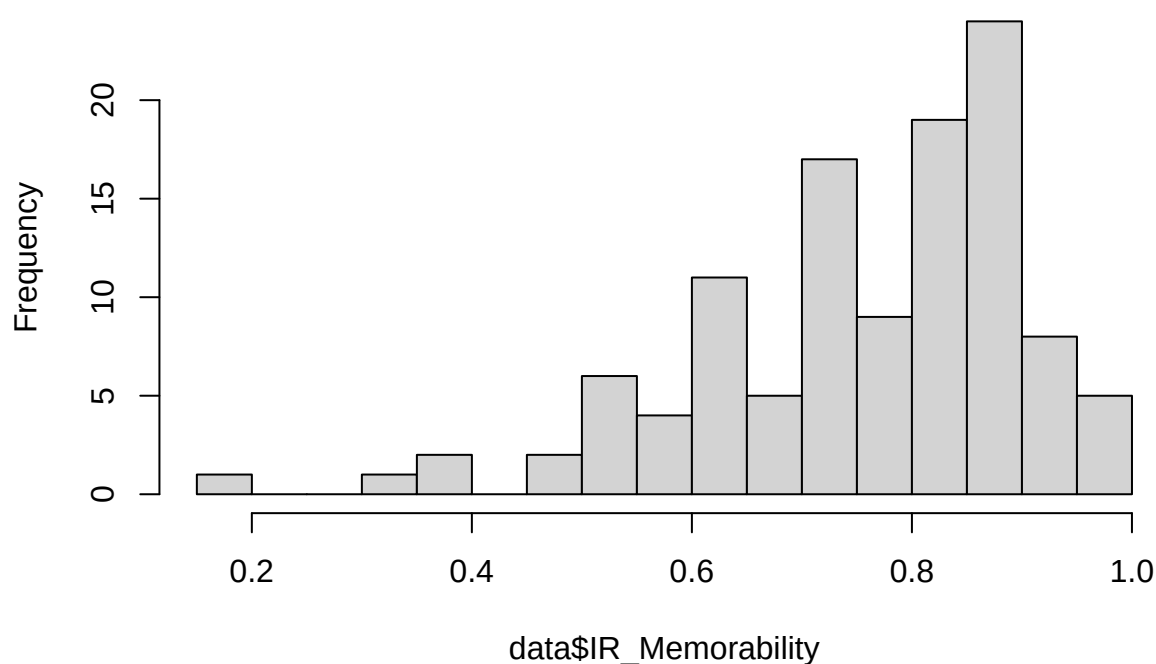
```
# Doing the above with the checks again showed that we cannot transform the data to become normal
```

```
# checking assumptions for beta regression - values should be bw 0 and 1 and acceptable shape - here it  
any(data$IR_Memorability <= 0 | data$IR_Memorability >= 1, na.rm = TRUE)
```

```
## [1] FALSE
```

```
hist(data$IR_Memorability, breaks = 20)
```

Histogram of data\$IR_Memorability



```
model <- betareg(IR_Memorability ~ avg_RT_IR + IR_Presses + Time_of_Day, data = data)

# removing highly correlated predictors using VIF

removed_vars <- c()
repeat {
  vif_values <- vif(model)
  threshold = 5

  if (is.matrix(vif_values)) {
    vif_values <- vif_values[, "GVIF^(1/(2*Df))"]
  }

  max_vif <- max(vif_values)

  if (max_vif <= threshold) break

  variable_to_remove <- names(which.max(vif_values))
  removed_vars <- c(removed_vars, variable_to_remove)
  formula_current <- formula(model)
  predictors <- attr(terms(formula_current), "term.labels")
  predictors <- predictors[predictors != variable_to_remove]
  response <- all.vars(formula_current)[1]
  new_formula <- as.formula(
    paste(response, "~", paste(predictors, collapse = " + "))
  )
}
```

```

model <- betareg(new_formula, data = data)
}

cat("Variables removed due to high VIF for model2:\n")

## Variables removed due to high VIF for model2:

print(removed_vars)

## NULL

# interpreting the model
cat("\nInterpretation\n")

##
## Interpretation

summary(model)

##
## Call:
## betareg(formula = IR_Memorability ~ avg_RT_IR + IR_Presses + Time_of_Day,
## data = data)
##
## Quantile residuals:
##      Min       1Q   Median       3Q      Max
## -3.1787 -0.4813  0.1078  0.5724  2.5195
##
## Coefficients (mean model with logit link):
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)   -1.1597776  0.3701858  -3.133  0.00173 **
## avg_RT_IR      -0.0004586  0.0001629  -2.815  0.00488 **
## IR_Presses       0.0809735  0.0045993  17.606 < 2e-16 ***
## Time_of_DayEvening 0.0091591  0.0969988   0.094  0.92477
## Time_of_DayNight -0.0762504  0.0941989  -0.809  0.41825
##
## Phi coefficients (precision model with identity link):
##      Estimate Std. Error z value Pr(>|z|)
## (phi)   36.278      4.772   7.601 2.93e-14 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Type of estimator: ML (maximum likelihood)
## Log-likelihood: 153 on 6 Df
## Pseudo R-squared: 0.6971
## Number of iterations: 25 (BFGS) + 2 (Fisher scoring)

confint(model)

```

```
##              2.5 %      97.5 %
## (Intercept)    -1.885328467 -0.4342267490
## avg_RT_IR      -0.000777944 -0.0001393139
## IR_Presses      0.071959066  0.0899878886
## Time_of_DayEvening -0.180955104  0.1992733163
## Time_of_DayNight -0.260876774  0.1083760166
## (phi)          26.923659350 45.6314868012
```

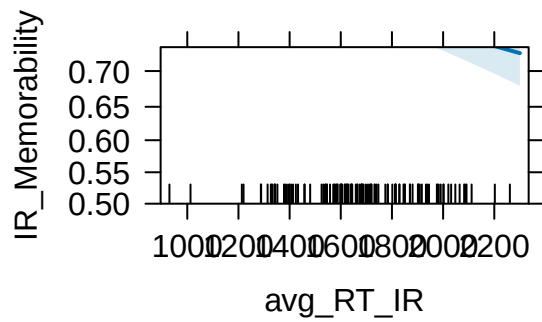
```
avg_slopes(model)
```

```
##
##      Term          Contrast Estimate Std. Error      z Pr(>|z|)      S
## IR_Presses dY/dX          1.33e-02  6.68e-04 19.9469 < 0.001 291.7
## Time_of_Day Evening - Day time 1.49e-03  1.58e-02  0.0944  0.92479   0.1
## Time_of_Day Night - Day time -1.26e-02  1.56e-02 -0.8114  0.41712   1.3
## avg_RT_IR dY/dX          -7.55e-05  2.68e-05 -2.8198  0.00481   7.7
##      2.5 %      97.5 %
##      0.012013  0.014631
##      -0.029391  0.032365
##      -0.043116  0.017869
##      -0.000128 -0.000023
##
## Type: response
```

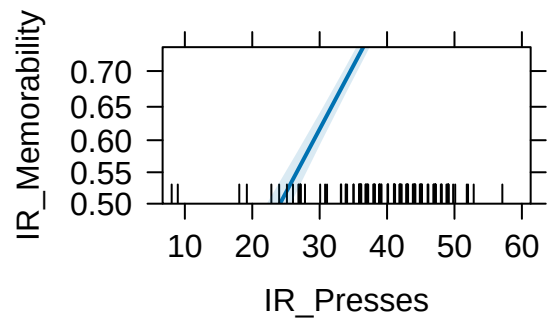
```
# Plotting the model
# The plot below shows how appropriate the model is for our data, and it's quite appropriate
# plot(model)
```

```
plot(allEffects(model), ylim = c(0, 1))
```

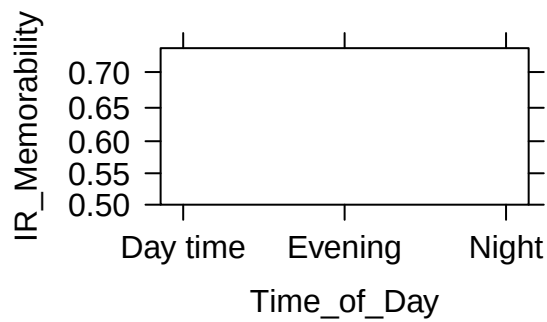
avg_RT_IR effect plot



IR_Presses effect plot

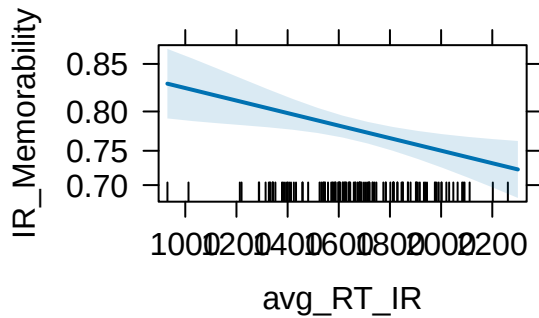


Time_of_Day effect plot

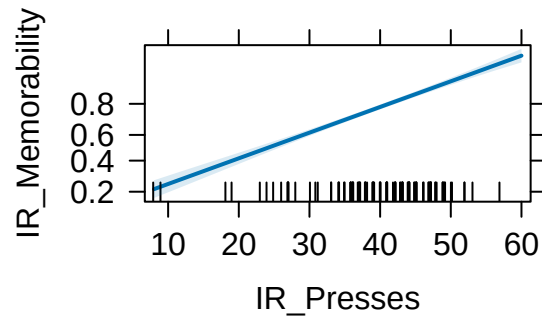


```
plot(allEffects(model))
```

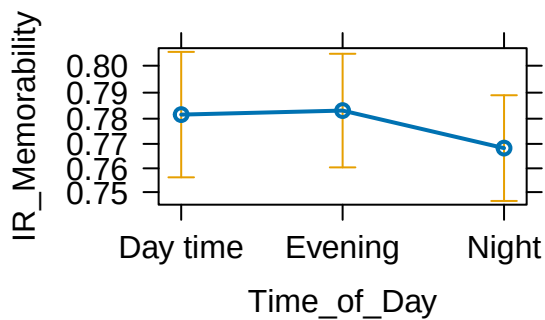
avg_RT_IR effect plot



IR_Presses effect plot



Time_of_Day effect plot



```
cat("AIC value of the model\n")
```

```
## AIC value of the model
```

```
AIC(model)
```

```
## [1] -293.9859
```

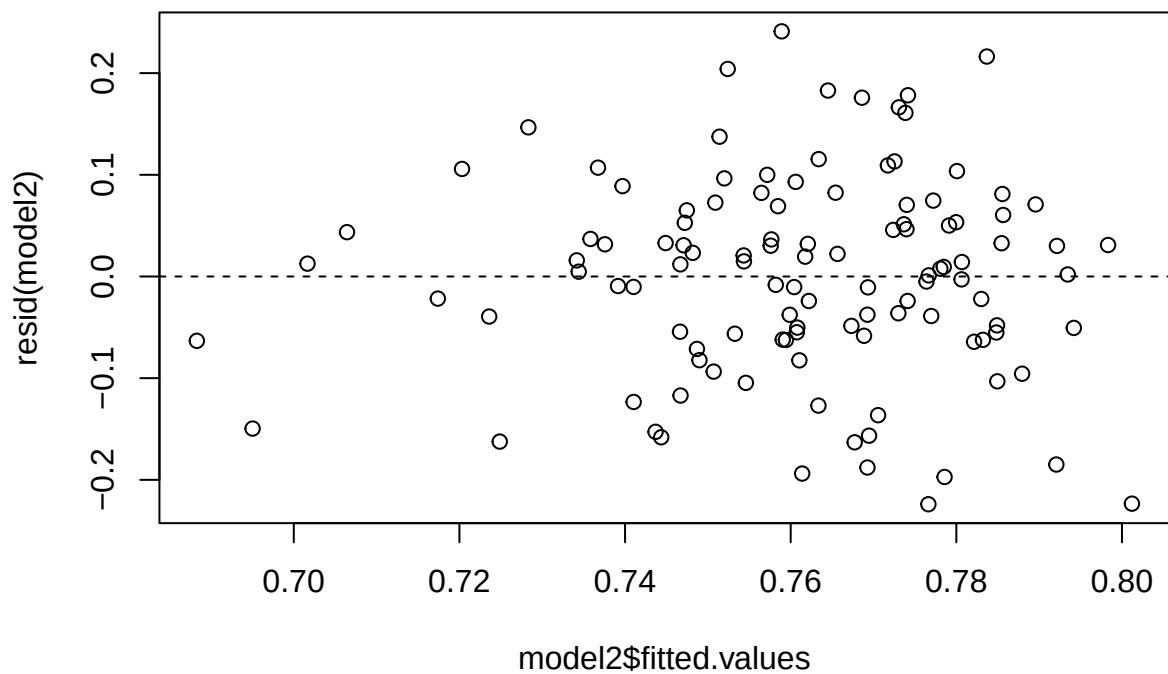
```
# Testing assumptions - WR Accuracy
```

```
model2 <- lm(WR_Accuracy ~ avg_RT_WR + WR_Presses + Time_of_Day, data = data)
```

```
# Linearity and homoskedasticity
```

```
plot(model2$fitted.values, resid(model2))
```

```
abline(h = 0, lty = 2)
```

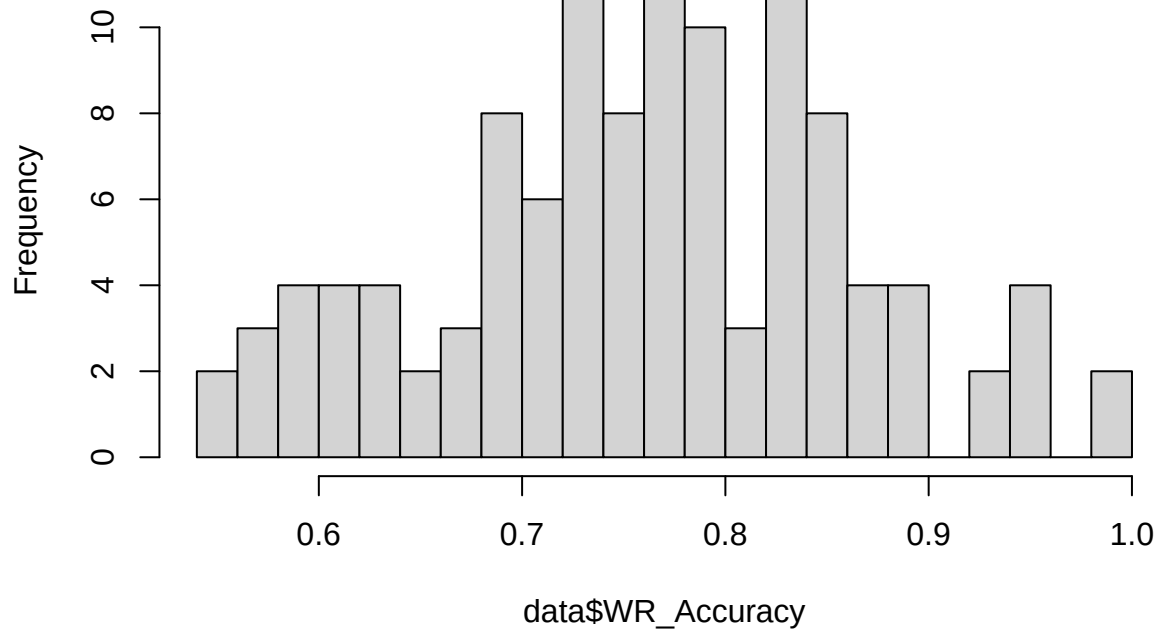



```
ncvTest(model2)
```

```
## Non-constant Variance Score Test  
## Variance formula: ~ fitted.values  
## Chisquare = 0.810557, Df = 1, p = 0.36796
```

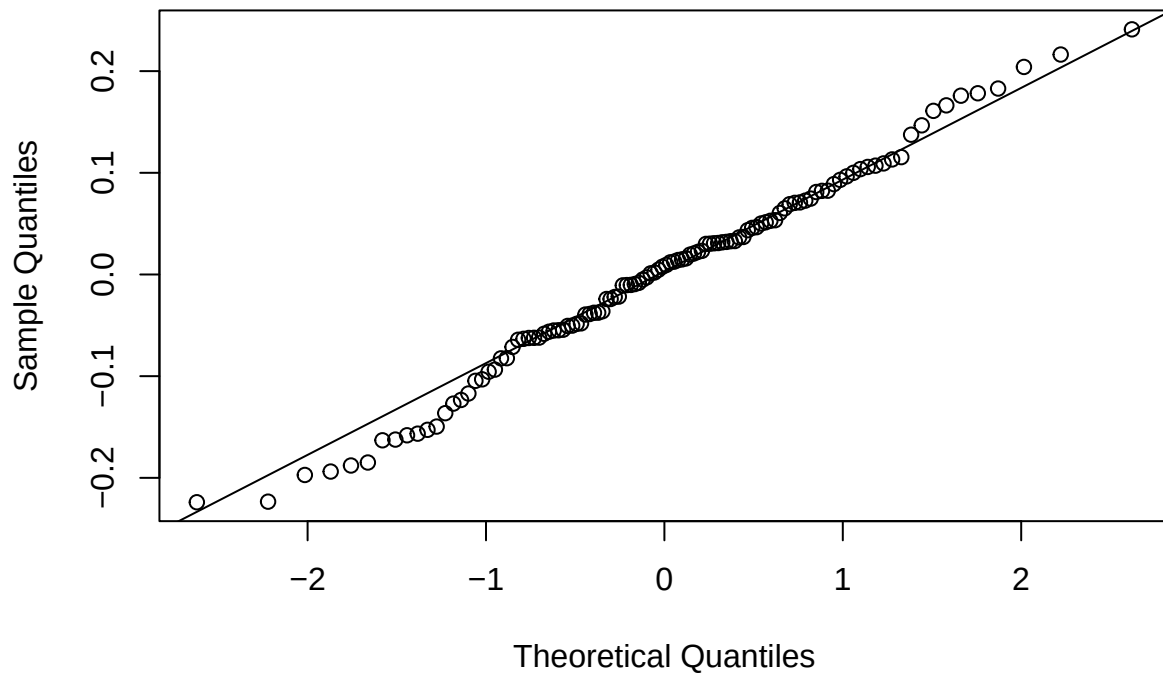
```
# Normality  
hist(data$WR_Accuracy, breaks = 20)
```

Histogram of data\$WR_Accuracy



```
qqnorm(resid(model2))  
qqline(resid(model2))
```

Normal Q-Q Plot

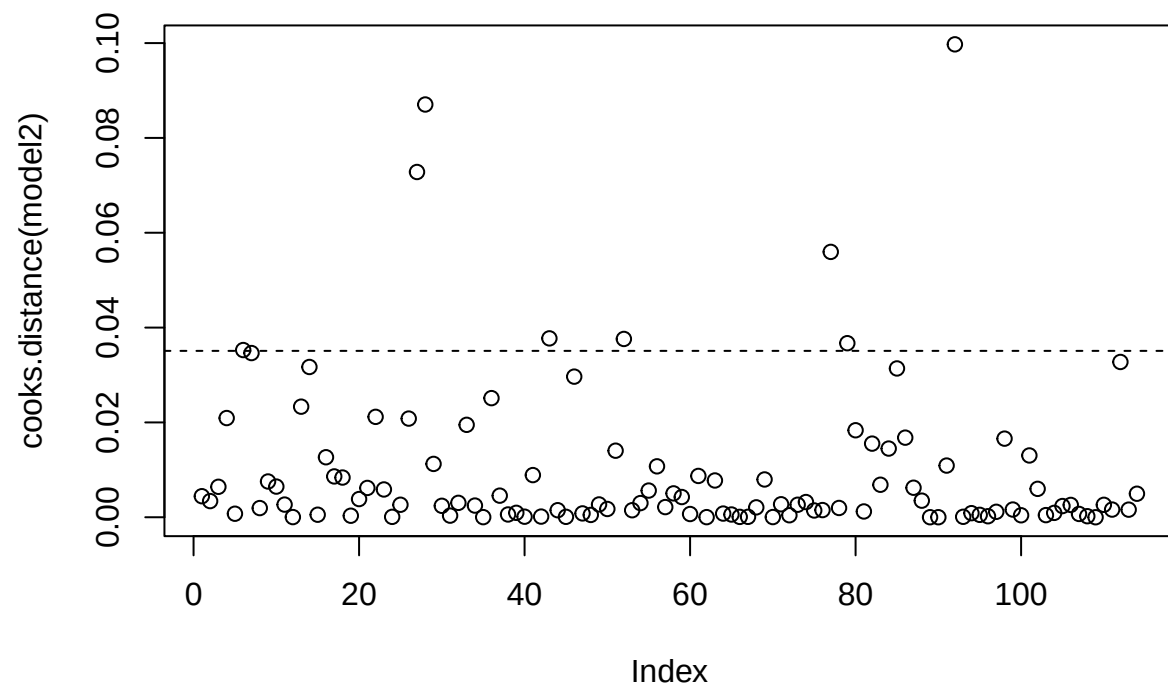


```
residuals_model <- residuals(model2)
ad.test(residuals_model)
```

```
##
## Anderson-Darling normality test
##
## data: residuals_model
## A = 0.33449, p-value = 0.5051
```

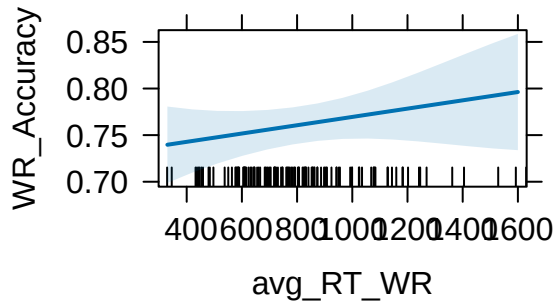
```
# Outliers
```

```
plot(cooks.distance(model2))
abline(h = 4/nrow(data), lty = 2)
```

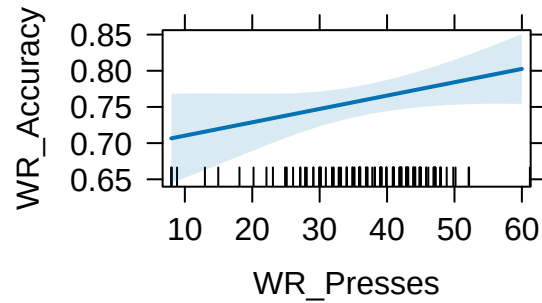


```
plot(allEffects(model2))
```

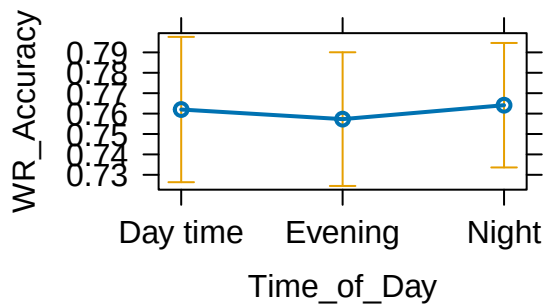
avg_RT_WR effect plot



WR_Presses effect plot



Time_of_Day effect plot



```
# Transform WR_Accuracy to (0,1) for beta regression because even though glm assumptions are satisfied,
n <- nrow(data)

data <- data %>%
  mutate(
    WR_Accuracy_adj = (WR_Accuracy * (n - 1) + 0.5) / n
  )

model2 <- betareg(WR_Accuracy_adj ~ avg_RT_WR + WR_Presses + Time_of_Day, data = data)

# removing highly correlated predictors using VIF

removed_vars <- c()
repeat {
  vif_values <- vif(model2)
  threshold = 5

  if (is.matrix(vif_values)) {
    vif_values <- vif_values[, "GVIF^(1/(2*Df))"]
  }

  max_vif <- max(vif_values)

  if (max_vif <= threshold) break

  variable_to_remove <- names(which.max(vif_values))
```

```

removed_vars <- c(removed_vars, variable_to_remove)
formula_current <- formula(model2)
predictors <- attr(terms(formula_current), "term.labels")
predictors <- predictors[predictors != variable_to_remove]
response <- all.vars(formula_current)[1]
new_formula <- as.formula(
  paste(response, "~", paste(predictors, collapse = " + "))
)

model2 <- betareg(new_formula, data = data)
}

```

```
cat("Variables removed due to high VIF for model2:\n")
```

```
## Variables removed due to high VIF for model2:
```

```
print(removed_vars)
```

```
## NULL
```

```

# interpreting the model
cat("\nInterpretation\n")

```

```

##
## Interpretation

```

```
summary(model2)
```

```

##
## Call:
## betareg(formula = WR_Accuracy_adj ~ avg_RT_WR + WR_Presses + Time_of_Day,
## data = data)
##
## Quantile residuals:
##      Min       1Q   Median       3Q      Max
## -1.9509 -0.5803 -0.0853  0.4537  4.2615
##
## Coefficients (mean model with logit link):
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)    0.4304783  0.3017356   1.427   0.1537
## avg_RT_WR       0.0002834  0.0002283   1.241   0.2146
## WR_Presses      0.0123532  0.0058405   2.115   0.0344 *
## Time_of_DayEvening 0.0504646  0.1472762   0.343   0.7319
## Time_of_DayNight 0.0667359  0.1396525   0.478   0.6327
##
## Phi coefficients (precision model with identity link):
##              Estimate Std. Error z value Pr(>|z|)
## (phi)    13.549      1.755      7.72 1.17e-14 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##

```

```
## Type of estimator: ML (maximum likelihood)
## Log-likelihood: 93.9 on 6 Df
## Pseudo R-squared: 0.03541
## Number of iterations: 20 (BFGS) + 2 (Fisher scoring)
```

```
confint(model2)
```

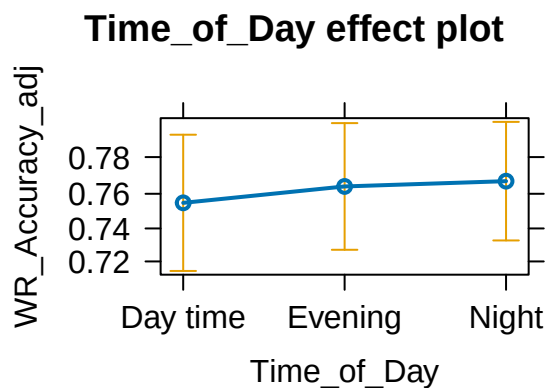
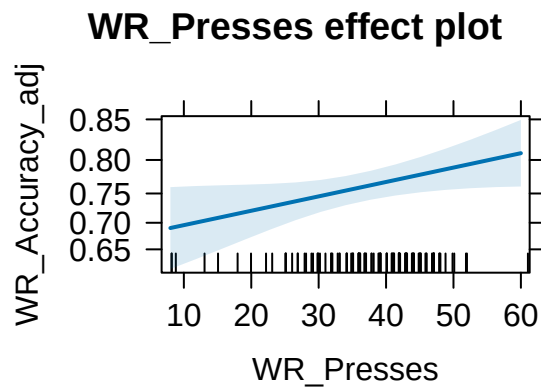
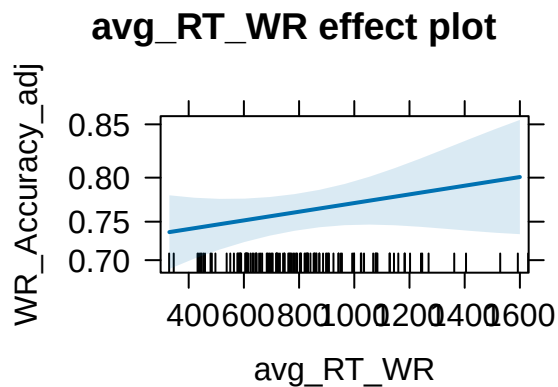
```
##              2.5 %      97.5 %
## (Intercept) -0.1609125666 1.021869e+00
## avg_RT_WR   -0.0001641494 7.308756e-04
## WR_Presses  0.0009059769 2.380048e-02
## Time_of_DayEvening -0.2381914244 3.391206e-01
## Time_of_DayNight -0.2069780233 3.404498e-01
## (phi)       10.1089371631 1.698879e+01
```

```
avg_slopes(model2)
```

```
##
##      Term          Contrast Estimate Std. Error    z Pr(>|z|)    S
## Time_of_Day Evening - Day time 9.21e-03  2.69e-02 0.342   0.7321 0.4
## Time_of_Day Night - Day time  1.21e-02  2.54e-02 0.477   0.6336 0.7
## WR_Presses dY/dX              2.23e-03  1.05e-03 2.121   0.0339 4.9
## avg_RT_WR dY/dX              5.12e-05  4.12e-05 1.242   0.2141 2.2
##      2.5 %   97.5 %
## -4.35e-02 0.061916
## -3.77e-02 0.061974
##  1.70e-04 0.004297
## -2.96e-05 0.000132
##
## Type: response
```

```
# Plotting the model
# The plot below shows how appropriate the model is for our data, and it's quite appropriate
# plot(model)
```

```
plot(allEffects(model2))
```



```
cat("AIC value of the model\n")
```

```
## AIC value of the model
```

```
AIC(model2)
```

```
## [1] -175.8078
```

```
# Beta regression on sentence level - for each sentence, taking word condition, sentence voice and inter
```

```
data2 <- raw_dataset %>%
  filter(isTarget == TRUE) %>%
  arrange(participant_ID, Stimulus, Timestamp) %>%
  mutate(
    # mapping conditions
    Word_Condition = case_when(
      str_detect(Stimulus, "^HH_") ~ "HH",
      str_detect(Stimulus, "^HVL_") ~ "HL",
      str_detect(Stimulus, "^LVH_") ~ "LH",
      str_detect(Stimulus, "^LVL_") ~ "LL"
    ),
    Word_Condition = factor(Word_Condition, levels = c("HH", "HL", "LH", "LL")),
    Sentence_Voice = case_when(
```



```

    str_detect(Stimulus, "_A$") ~ "Active",
    str_detect(Stimulus, "_P$") ~ "Passive",
    TRUE ~ NA_character_
  ),
  Sentence_Voice = factor(Sentence_Voice, levels = c("Active", "Passive")),

  Accuracy_IR = suppressWarnings(as.numeric(Accuracy_IR)),
  Accuracy_WR = suppressWarnings(as.numeric(Accuracy_WR)),

  # store previous IR result
  prev_IR = lag(Accuracy_IR)
) %>%
group_by(Stimulus, Word_Condition, Sentence_Voice) %>%
summarise(
  Participants = n_distinct(participant_ID),
  # IR components
  Total_Repeats = sum(isRepeat == TRUE & Event == "Sentence shown", na.rm = TRUE),
  Total_Denom = sum(is.na(isRepeat) & Event == "Sentence shown"),
  Hits = sum(Event == "IR pressed" & isRepeat == TRUE & Accuracy_IR == 1, na.rm = TRUE),
  False_Alarms = sum(Event == "IR pressed" & is.na(isRepeat), na.rm = TRUE),

  # IR metrics
  Hit_Rate = if_else(Total_Repeats > 0, Hits / Total_Repeats, 0),
  False_Alarm_Rate = if_else(Total_Denom > 0, False_Alarms / Total_Denom, 0),
  IR_Memorability = Hit_Rate - False_Alarm_Rate,

  # WR components (after correct IR)
  WR_correct = sum(Event == "WR pressed" & Accuracy_WR == 1 & prev_IR == 1, na.rm = TRUE),
  WR_total = sum(Event == "WR pressed" & prev_IR == 1, na.rm = TRUE),

  # WR metric
  WR_Accuracy = if_else(WR_total > 0, WR_correct / WR_total, 0),

  .groups = "drop"
) %>%
mutate(WR_Accuracy = replace_na(WR_Accuracy, 0))

data2 <- data2 %>%
  filter(Participants >= 15)

```

```

# beta regression for IR memorability

n <- nrow(data2)

data2 <- data2 %>%
  mutate(
    IR_Memorability_adj = (IR_Memorability * (n - 1) + 0.5) / n
  )

model3 <- betareg(IR_Memorability_adj ~ Word_Condition*Sentence_Voice, data = data2)

# interpreting the model

```

```
cat("\nInterpretation\n")
```

```
##  
## Interpretation
```

```
summary(model3)
```

```
##  
## Call:  
## betareg(formula = IR_Memorability_adj ~ Word_Condition * Sentence_Voice,  
##       data = data2)  
##  
## Quantile residuals:  
##      Min      1Q   Median      3Q      Max  
## -2.8455 -0.6217 -0.0907  0.3707  3.5921  
##  
## Coefficients (mean model with logit link):  
##  
##              Estimate Std. Error z value Pr(>|z|)  
## (Intercept)      1.32596    0.12905  10.274   <2e-16 ***  
## Word_ConditionHL -0.05069    0.18227  -0.278    0.781  
## Word_ConditionLH -0.03691    0.17903  -0.206    0.837  
## Word_ConditionLL -0.24803    0.17531  -1.415    0.157  
## Sentence_VoicePassive -0.27895    0.17595  -1.585    0.113  
## Word_ConditionHL:Sentence_VoicePassive  0.22644    0.25229    0.898    0.369  
## Word_ConditionLH:Sentence_VoicePassive  0.13635    0.25000    0.545    0.585  
## Word_ConditionLL:Sentence_VoicePassive  0.38184    0.24692    1.546    0.122  
##  
## Phi coefficients (precision model with identity link):  
##      Estimate Std. Error z value Pr(>|z|)  
## (phi)   6.8110    0.5198   13.1   <2e-16 ***  
## ---  
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1  
##  
## Type of estimator: ML (maximum likelihood)  
## Log-likelihood: 189.4 on 9 Df  
## Pseudo R-squared: 0.009906  
## Number of iterations: 19 (BFGS) + 2 (Fisher scoring)
```

```
confint(model3)
```

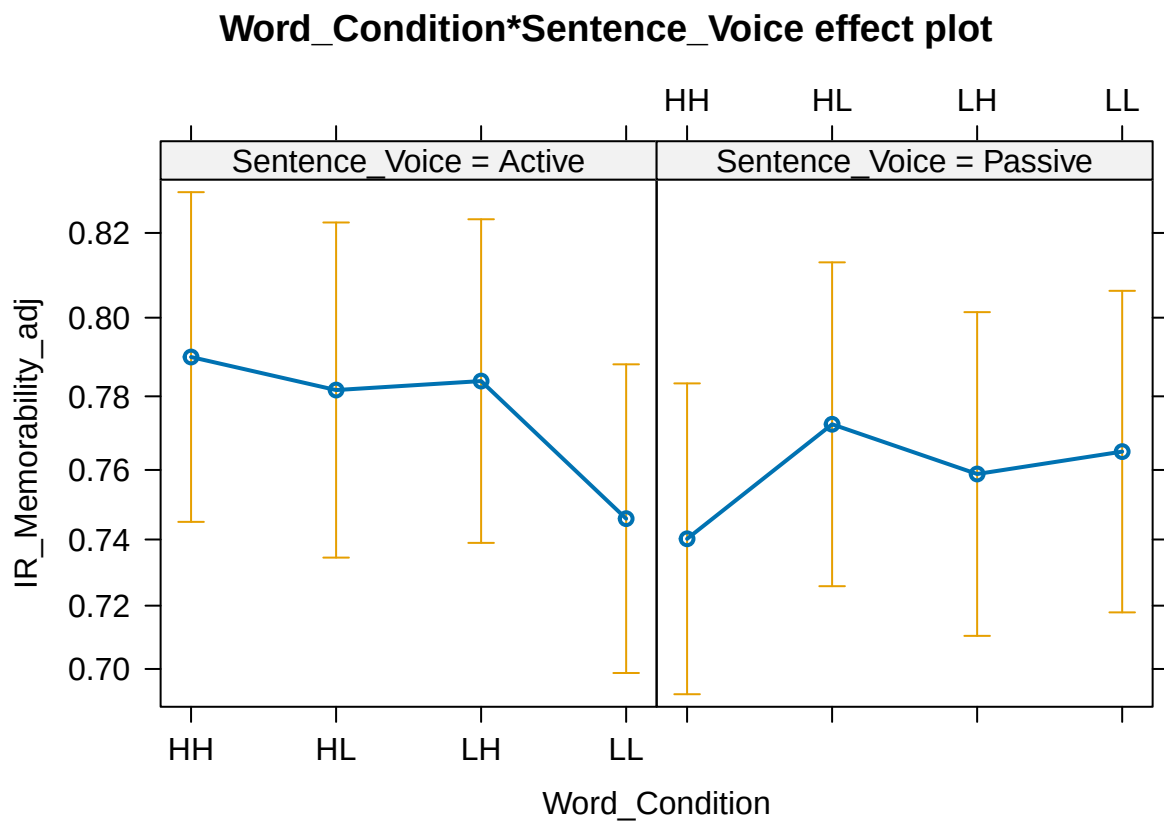
```
##              2.5 %      97.5 %  
## (Intercept)   1.0730140  1.57889787  
## Word_ConditionHL -0.4079230  0.30654910  
## Word_ConditionLH -0.3878024  0.31398660  
## Word_ConditionLL -0.5916346  0.09557641  
## Sentence_VoicePassive -0.6237972  0.06590611  
## Word_ConditionHL:Sentence_VoicePassive -0.2680322  0.72091624  
## Word_ConditionLH:Sentence_VoicePassive -0.3536499  0.62634088  
## Word_ConditionLL:Sentence_VoicePassive -0.1021121  0.86579143  
## (phi)         5.7921667  7.82987310
```

```
avg_slopes(model3)
```

```
##
##           Term           Contrast Estimate Std. Error      z Pr(>|z|)    S
## Sentence_Voice Passive - Active -0.01623    0.0158 -1.030   0.303 1.7
## Word_Condition HL - HH          0.01185    0.0222  0.534   0.593 0.8
## Word_Condition LH - HH          0.00620    0.0222  0.279   0.780 0.4
## Word_Condition LL - HH         -0.00969    0.0224 -0.433   0.665 0.6
##   2.5 % 97.5 %
## -0.0471 0.0147
## -0.0316 0.0553
## -0.0373 0.0497
## -0.0536 0.0342
##
## Type: response
```

```
# Plotting the model
# The plot below shows how appropriate the model is for our data, and it's quite appropriate
# plot(model)
```

```
plot(allEffects(model3))
```



```
cat("AIC value of the model\n")
```

```
## AIC value of the model
```

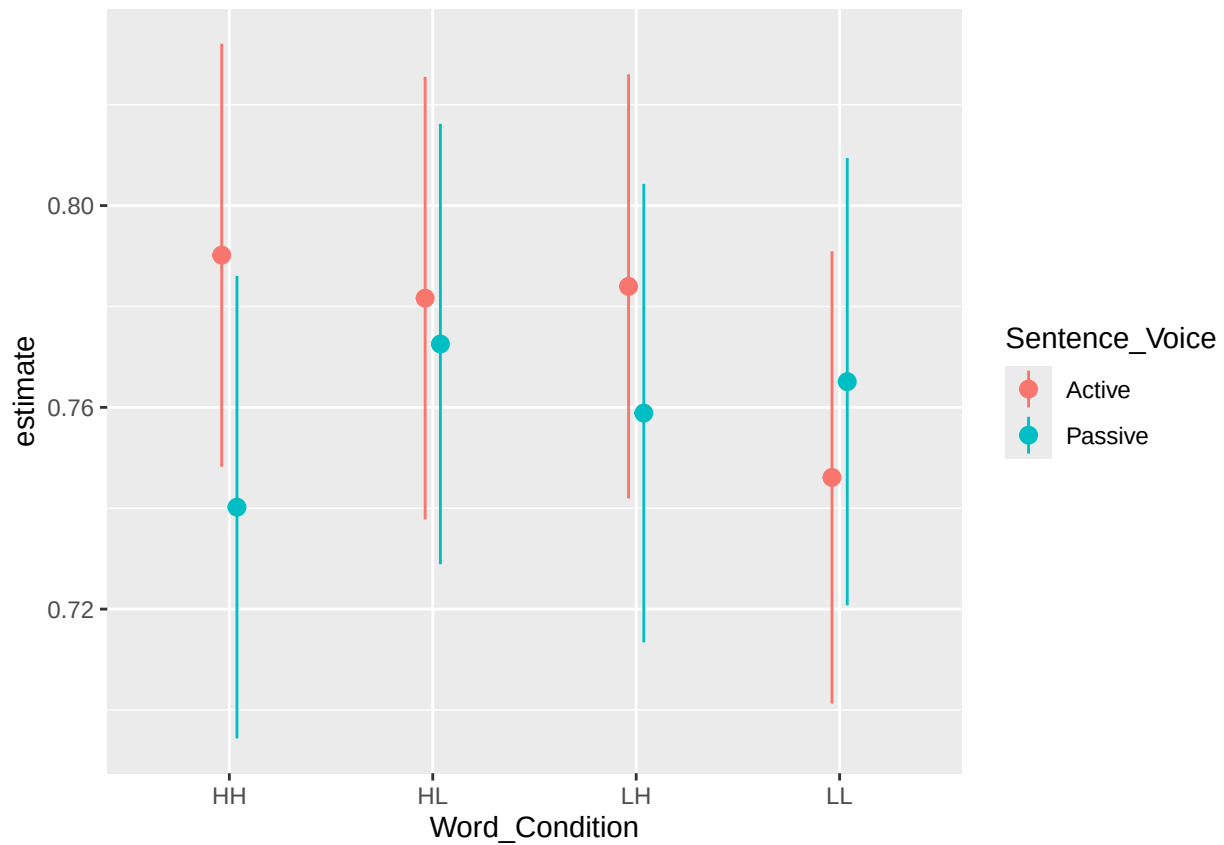
```
AIC(model3)
```

```
## [1] -360.8489
```

```
cat("Plotting Predictions\n")
```

```
## Plotting Predictions
```

```
plot_predictions(model3, condition = c("Word_Condition", "Sentence_Voice"))
```



```
# beta regression for WR_Accuracy
```

```
n <- nrow(data2)
```

```
data2 <- data2 %>%
```

```
  mutate(
```

```
    WR_Accuracy_adj = (WR_Accuracy * (n - 1) + 0.5) / n
```

```
  )
```

```
model4 <- betareg(WR_Accuracy_adj ~ Word_Condition*Sentence_Voice, data = data2)
```

```
# interpreting the model
cat("\nInterpretation\n")
```

```
##
## Interpretation
```

```
summary(model4)
```

```
##
## Call:
## betareg(formula = WR_Accuracy_adj ~ Word_Condition * Sentence_Voice,
## data = data2)
##
## Quantile residuals:
##      Min       1Q   Median       3Q      Max
## -1.8886 -0.6927 -0.2107  0.2822  3.0024
##
## Coefficients (mean model with logit link):
##
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)      1.14413    0.14591   7.841 4.45e-15 ***
## Word_ConditionHL      0.13408    0.20802   0.645  0.5192
## Word_ConditionLH      0.51075    0.20813   2.454  0.0141 *
## Word_ConditionLL      0.19970    0.20350   0.981  0.3264
## Sentence_VoicePassive  0.09771    0.20365   0.480  0.6314
## Word_ConditionHL:Sentence_VoicePassive -0.13864    0.29177  -0.475  0.6347
## Word_ConditionLH:Sentence_VoicePassive -0.50532    0.29285  -1.726  0.0844 .
## Word_ConditionLL:Sentence_VoicePassive -0.50035    0.28668  -1.745  0.0809 .
##
## Phi coefficients (precision model with identity link):
##      Estimate Std. Error z value Pr(>|z|)
## (phi)   4.3847    0.3358  13.06  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Type of estimator: ML (maximum likelihood)
## Log-likelihood: 178.3 on 9 Df
## Pseudo R-squared: 0.02795
## Number of iterations: 24 (BFGS) + 2 (Fisher scoring)
```

```
confint(model4)
```

```
##              2.5 %      97.5 %
## (Intercept)   0.8581561 1.43010528
## Word_ConditionHL -0.2736352 0.54178724
## Word_ConditionLH  0.1028254 0.91867751
## Word_ConditionLL -0.1991486 0.59854218
## Sentence_VoicePassive -0.3014360 0.49684973
## Word_ConditionHL:Sentence_VoicePassive -0.7104976 0.43322100
## Word_ConditionLH:Sentence_VoicePassive -1.0792878 0.06864401
```

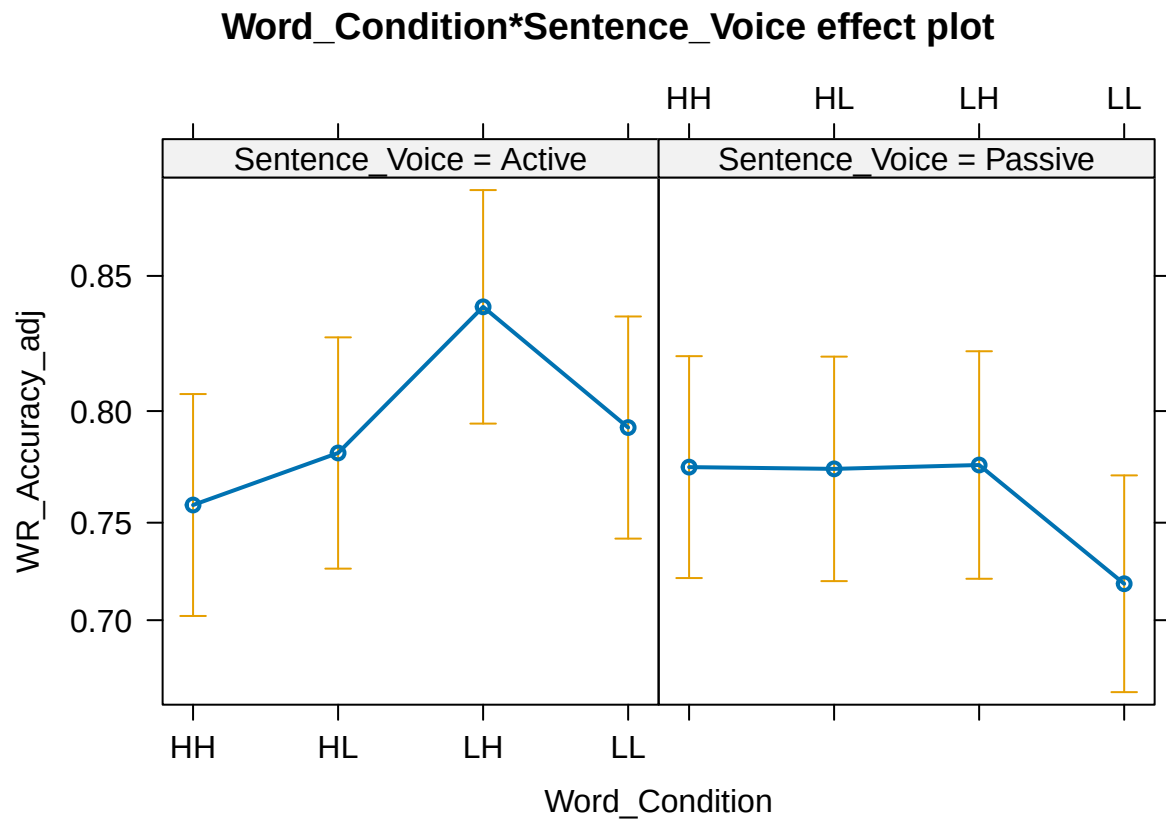
```
## Word_ConditionLL: Sentence_VoicePassive -1.0622265 0.06153211
## (phi) 3.7264420 5.04289249
```

```
avg_slopes(model4)
```

```
##
##          Term          Contrast Estimate Std. Error      z Pr(>|z|)    S
## Sentence_Voice Passive - Active  -0.0318     0.0178 -1.788   0.0737 3.8
## Word_Condition HL - HH           0.0115     0.0256  0.449   0.6533 0.6
## Word_Condition LH - HH           0.0412     0.0244  1.686   0.0918 3.4
## Word_Condition LL - HH          -0.0108     0.0259 -0.416   0.6774 0.6
##      2.5 % 97.5 %
## -0.06664 0.00305
## -0.03866 0.06165
## -0.00668 0.08899
## -0.06163 0.04005
##
## Type: response
```

```
# Plotting the model
# The plot below shows how appropriate the model is for our data, and it's quite appropriate
# plot(model)
```

```
plot(allEffects(model4))
```



```
cat("AIC value of the model\n")
```

```
## AIC value of the model
```

```
AIC(model4)
```

```
## [1] -338.6919
```

```
cat("Plotting Predictions\n")
```

```
## Plotting Predictions
```

```
plot_predictions(model4, condition = c("Word_Condition", "Sentence_Voice"))
```

